



Licence de Mécanique

LU3MEE01 - Systèmes Robotiques S2-25

Rapport de Projet

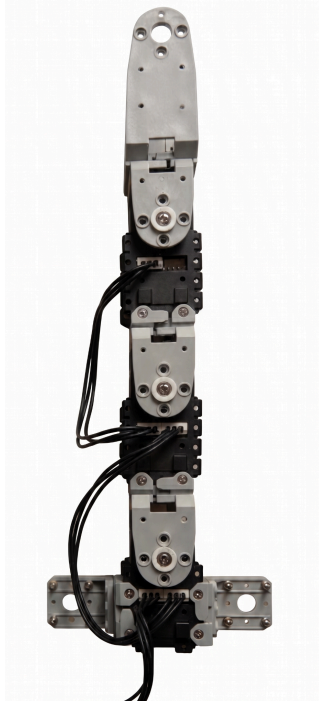
Réalisation expérimentale d'un robot sériel

CROZETIERE Etienne 21304820

VELASCO Matteo 21423434

0.Introduction

0.1 Architecture Générale du Robot



Le dispositif expérimental consiste en un robot sériel de type "bras 3R plan" fonctionnant dans un plan bidimensionnel. Il est constitué d'une chaîne cinématique de trois articulations animées par des moteurs identiques montés en série.

L'ensemble de la structure est ancré sur un support fixe (bâti) et se termine par un organe terminal dont l'objectif est d'exécuter des tâches de positionnement ou de suivi de trajectoire (comme l'écriture ou la manipulation de pièces).

Figure 1 : Bras Robotique

0.2. Les Actionneurs : Servomoteurs DYNAMIXEL AX12

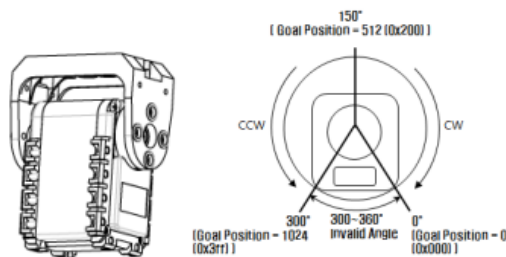


FIGURE 1 – (gauche) Moteur AX12. (droite) Convention pour le repérage des positions angulaires du moteur.

Le mouvement du bras est assuré par des servomoteurs intelligents. Leurs caractéristiques majeures sont les suivantes :

- Module tout-en-un : Chaque moteur intègre à la fois le moteur à courant continu, un réducteur (rapport 254:1), un contrôleur local et un pilote de communication.

- Contrôle en Position : Les moteurs sont asservis individuellement. Ils reçoivent des consignes absolues codées sous forme de "pas moteur" variant de 0 à 1023, ce qui correspond à une plage angulaire utile de 300°.
- Retour d'information : En temps réel et de manière non bloquante pour le code, chaque actionneur peut renvoyer sa position courante, sa vitesse de rotation instantanée, sa charge ou encore sa température.
- Identifiant Unique : Bien que les moteurs soient rigoureusement identiques, un numéro d'identification unique (ID) inscrit sur leur châssis permet de s'adresser sélectivement à un moteur précis au sein de la chaîne.

0.3. La Chaîne de Communication et d'Alimentation

La communication et la puissance électrique du robot reposent sur une architecture spécifique :

- Connexion en série : Les moteurs sont raccordés les uns à la suite des autres à l'aide de câbles à 3 broches transmettant les signaux et la puissance.
- Interface U2D2 / USB2DYNAMIXEL : Ce boîtier sert de pont entre l'ordinateur et le robot.



Figure 2 : Boîtier USB2DYNAMIXEL

- Alimentation dédiée : Le système est alimenté par un bloc externe indispensable pour fournir le couple nécessaire aux moteurs (jusqu'à 1,52 Nm).

0.4. Environnement Logiciel (Pilote MATLAB)

Le contrôle global du robot est centralisé sur un ordinateur via MATLAB.

Partie 1 : Montage, caractérisation et commande des actionneurs

Nous allons étudier dans un premier temps la réponse temporelle des actionneurs en analysant l'évolution de leur position et de leur vitesse pour différentes consignes. Cette étape permet notamment de confronter la vitesse mesurée par les capteurs internes filtrés à celle estimée mathématiquement par la méthode des différences finies, afin d'en évaluer les limites et d'analyser l'impact du bruit de quantification numérique. Enfin, l'évaluation de la répétabilité finale du positionnement permet de quantifier la précision globale de l'actionneur avant son intégration complète dans la structure robotique 3R plane.

1.1 Step en Radian

Relation mathématiques reliant les pas moteur lues depuis l'actionneur en valeur angulaire (radian) :

$$q = ((\text{step-offset})/(512/(150*\pi/180)))$$

La valeur offset permet de régler le 0 de la mesure.

1.2 Radian en Step

Relation inverse permettant de relier l'angle en radian à la valeur de pas moteur :

$$p = \text{offset} + q*(180*1023)/(300*\pi)$$

1.3 Vitesse moyenne

Pour un mouvement allant de 0 à 1023 pas moteur, voici le graphique de la vitesse moyenne en fonction du temps. En pas moteur et en radian :

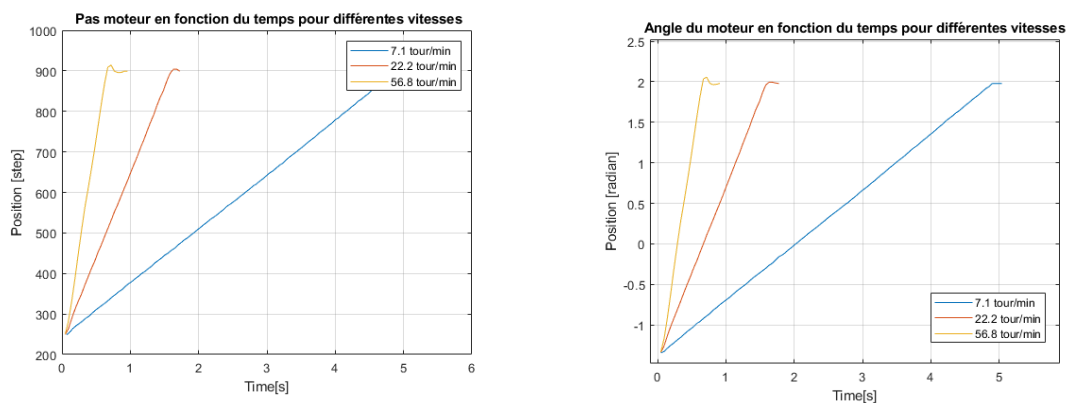


Figure 3 : Vitesse moyenne en fonction du temps en pas moteur puis en radian

En faisant varier la consigne de vitesse, on observe graphiquement que la pente de la courbe de position change : elle croît proportionnellement avec la vitesse injectée. Les résultats montrent que la vitesse réelle suit fidèlement la consigne imposée durant la phase de régime permanent. L'écart minime observé s'explique par les brèves phases d'accélération et de décélération aux extrémités de la trajectoire, durant lesquelles le moteur n'a pas encore atteint ou maintient sa vitesse de croisière.

1.4 Evolution de la vitesse pour différentes positions

Ici on trouve une vitesse variant autour de 7 tour/min ce qui correspond à la vitesse mise sur le programme de 64 qui converti avec val2speed nous donne 7.1 tour/ min

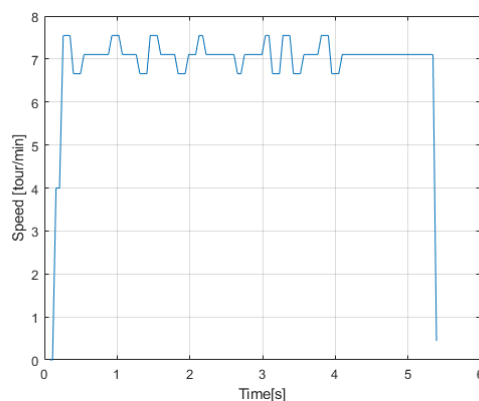


Figure 4 : Evolution de la vitesse de la position moteur 250 à 950

Le tracé direct de la vitesse lue via `getSpeed` en fonction du temps met en évidence le profil de vitesse de l'actionneur. On y distingue clairement trois phases distinctes : une accélération transitoire très rapide, un plateau stabilisé correspondant au régime permanent (vitesse de croisière), puis une décélération brutale lorsque le moteur approche de sa position cible (`pos_final = 900`).

En comparant le plateau stabilisé avec la consigne `setSpeed` (convertie en tr/min), les valeurs concordent de manière très précise. Le moteur régule efficacement sa vitesse grâce à son asservissement interne. Les légères fluctuations visibles sur le plateau mesuré proviennent du bruit de quantification du capteur de vitesse et des variations mineures de la tension d'alimentation sous charge.

1.5 Différences finies

Estimation de la vitesse par différence finie :

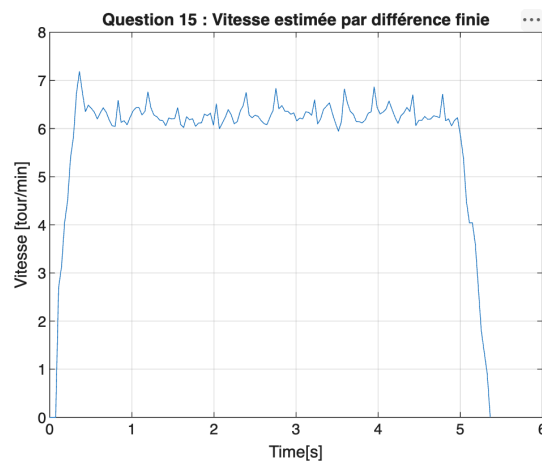


Figure 5 : Evolution de la vitesse par différence finie de la position moteur 250 à 950

On observe que la courbe de vitesse calculée par différence finie présente des micro-oscillations et des petites irrégularités continues tout au long du plateau de déplacement. Cela est dû au fait que le capteur du moteur quantifie la position sous forme de pas entiers discrets.

1.6 Essais Successifs

En effectuant plusieurs essais successifs identiques on évalue la répétabilité des caractéristiques du mouvement.

```
pos_depart= 250  
pos_final= 900  
vitesse = (64,128,256,512)
```

- Pour une vitesse de 512 on a une moyenne d'erreur de 2
- Pour une vitesse de 256 on a une moyenne d'erreur de 1.6667
- Pour une vitesse de 128 on a une moyenne d'erreur de 1
- Pour une vitesse de 64 on a une moyenne d'erreur de 0.333

Les essais successifs mettent en évidence une corrélation directe entre la vitesse de déplacement et l'erreur statique de positionnement final. Plus la vitesse de consigne est élevée, plus l'erreur moyenne en fin de mouvement augmente.

À haute vitesse, l'inertie propre des segments du bras entraîne un léger dépassement ou force le correcteur interne de l'actionneur à couper le couple brusquement pour ne pas franchir la cible, laissant le moteur s'arrêter juste avant sa position idéale à cause des frottements secs. À l'inverse, une vitesse faible (comme 64) permet un accostage en douceur, ce qui maximise la précision angulaire finale et assure une excellente répétabilité (erreur quasi nulle de 0.33 pas).

1.7 Conclusion

Cette première partie a permis de caractériser expérimentalement les servomoteurs Dynamixel AX12 d'un robot sériel. Les mesures ont validé l'efficacité de l'asservissement en position, montrant que la vitesse réelle suit proportionnellement la consigne imposée.

De plus, la comparaison des méthodes de calcul a mis en évidence l'impact du bruit de quantification : l'estimation de la vitesse par différence finie engendre de fortes micro-oscillations dues au retour de position discret (en pas entiers), contrairement au signal fluide et filtré fourni directement par le capteur interne du moteur. Ce travail souligne l'importance de prendre en compte les limitations matérielles et le traitement des signaux lors du pilotage d'une maquette réelle.

Partie 2 : génération de mouvements et commande

2.1 Schéma cinématique d'un robot 3R plan

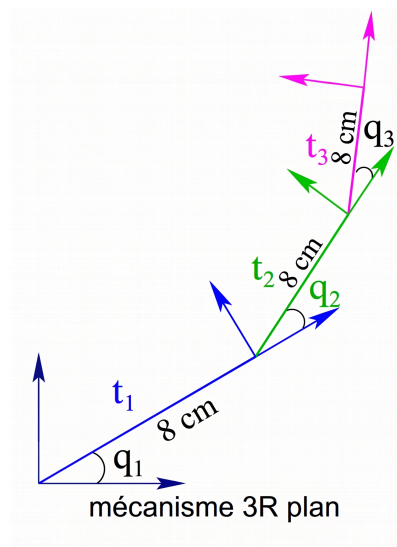


Figure 6 : Représentation cinématique d'un mécanisme 3R

Les angles sont q_1 , q_2 , q_3 avec des longueurs des segments t_1, t_2, t_3 de 8 cm chacune.

2.2 Déclaration du modèle et affichage du bras robotique

```
% Paramètres de la communications série
PORT = 11;
BAUD = 1000000;% A modifier avec le n° du port COM utilisé
%%
ID = 3;
% ID du moteur terminal
segment=[0.5,0.5,0.5];
modele = robot('modele',segment); % Connexion aux moteurs
maquette = robot('real',PORT,BAUD); % Connexion aux moteurs
%%

pos = 700.18;
maquette.setStepPosition(ID,pos);

p1 = maquette.getStepPosition(1);
q1 = step2angle(p1,512);
p2 = maquette.getStepPosition(2);
q2 = step2angle(p2,512);
p3 = maquette.getStepPosition(3);
q3 = step2angle(p3,512);
modele.setAngularPosition([q1,q2,q3]);
modele.plot(0,"red",0);
```

On déclare le *modele* qui nous permet de modéliser notre robot sur matlab. Et on l'affiche via la commande *modele.plot*.

Figure 8: Partie de code

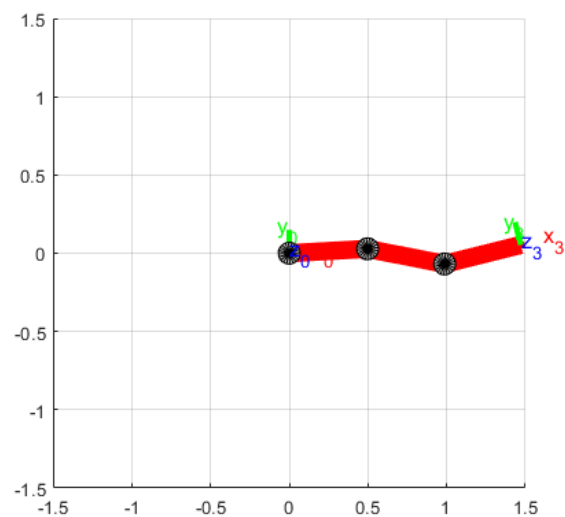


Figure 8: Position du modèle

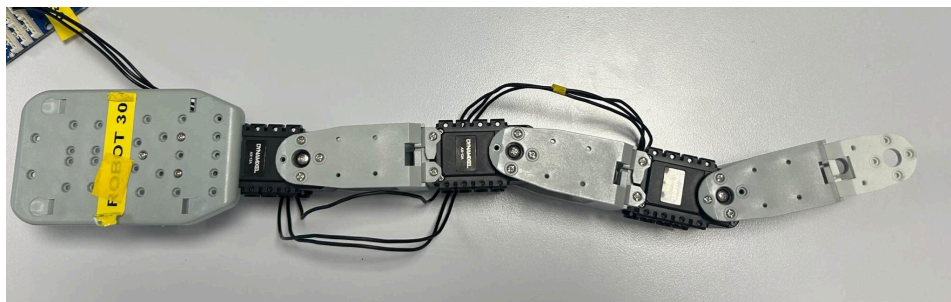


Figure 9 : Position réel

On observe que la position réelle est parfaitement alignée avec la position du modèle.

2.3 Principe du Modèle Géométrique Direct (MGD)

Le Modèle géométrique Direct (MGD) est la relation mathématique qui permet de calculer la position cartésienne (x,y) du bout du bras dans l'espace, à partir des angles des moteurs (q1,q2,q3).

Pour notre robot plan à trois articulations, le MGD est obtenu par trigonométrie en projetant la longueur de chaque segment (L1,L2,L3) sur les axes X et Y :

$$x = L_1 \cos(q_1) + L_2 \cos(q_1 + q_2) + L_3 \cos(q_1 + q_2 + q_3)$$

$$y = L_1 \sin(q_1) + L_2 \sin(q_1 + q_2) + L_3 \sin(q_1 + q_2 + q_3)$$

L'intégration de ces équations dans la fonction MATLAB `mod_geo_dir` permet de traduire instantanément les configurations articulaires des moteurs en coordonnées millimétriques. La correspondance entre les positions mesurées sur le robot réel et celles calculées en simulation valide l'exactitude des équations théoriques du modèle.

```
q4 = [300,400,550];
q5 = [400,450,300];
q6 = [512,512,512];
maquette.setSpeed(1,32);
maquette.setSpeed(2,32);
maquette.setSpeed(3,32);
val_min = 50;
val_max = 950;

%%
modele.setAngularPosition(q4);
T = modele.mod_geo_dir();

x1 = T(1,4);
y1 = T(2,4);
theta1 = inv_rotation_Z_etu(rotation_Z_etu(step2angle(q4(3),512)));
p1n = limit(q4,val_min,val_max);
maquette.setStepPosition([1,2,3],p1n);
modele.setAngularPosition(step2angle(q4,512));
pause(3);

%%
modele.setAngularPosition(q5);
T = modele.mod_geo_dir();
x2 = T(1,4);
y2 = T(2,4);
theta2 = inv_rotation_Z_etu(rotation_Z_etu(step2angle(q5(3),512)));
p2n = limit(q5,val_min,val_max);
maquette.setStepPosition([1,2,3],p2n);
modele.setAngularPosition(step2angle(q5,512));
pause(3);

%%Validation des valeurs numériques sur le modèle physique
modele.setAngularPosition(q6);
T = modele.mod_geo_dir();
x3 = T(1,4);
y3 = T(2,4);
theta3 = inv_rotation_Z_etu(rotation_Z_etu(step2angle(q6(3),512)));
p3n = limit(q6,val_min,val_max);
maquette.setStepPosition([1,2,3],p3n);
modele.setAngularPosition(step2angle(q6,512));
pause(3);
```

Figure 10: Partie de code

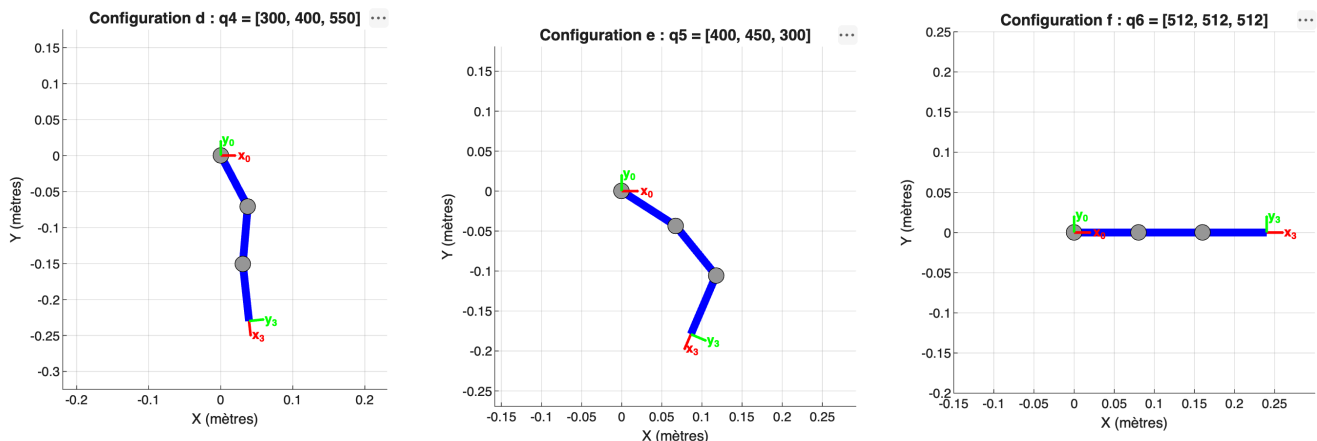


Figure 11: Représentation du robot pour 3 configurations différentes

Sans interpolation, chaque moteur se déplace indépendamment vers sa consigne articulaire finale. La trajectoire de l'organe terminal dans l'espace opérationnel est donc courbe et non maîtrisée : on contrôle uniquement la pose finale, pas le chemin parcouru.

2.4 Principe du Modèle Géométrique inverse (MGI)

Le Modèle Géométrique Inverse (MGI) fait exactement l'inverse du MGD : il permet de trouver les angles des moteurs (q_1, q_2, q_3) nécessaires pour amener le bout du bras à une position (x,y) bien précise.

Contrairement au MGD qui n'a qu'une seule réponse, le MGI peut trouver plusieurs solutions pour une même position cible (par exemple, le robot peut atteindre un point avec le "coude" vers le haut ou vers le bas).

Cette fonction est indispensable pour programmer le robot : c'est elle qui traduit un objectif de déplacement en millimètres en consignes d'angles que les moteurs Dynamixel peuvent exécuter.

```
val_min = 50;
val_max = 950;
cibleA = [1; 1; pi/4];
[sol1, sol2] = modele.mod_geo_inv(cibleA(1), cibleA(2), cibleA(3));

sol1_steps = angle2step(sol1, 512);
sol2_steps = angle2step(sol2, 512);

sol1_ok = all(sol1_steps >= val_min & sol1_steps <= val_max);
sol2_ok = all(sol2_steps >= val_min & sol2_steps <= val_max);

if sol1_ok
    q_choisi_rad = sol1;
    q_choisi_steps = sol1_steps;
elseif sol2_ok
    q_choisi_rad = sol2;
    q_choisi_steps = sol2_steps;
else
    error('Aucune solution');
end

modele.setAngularPosition(q_choisi_rad);
T_verif = modele.mod_geo_dir();
%modele.plot(0, 'red', 0);

maquette.setStepPosition([1, 2, 3], q_choisi_steps);
```

Figure 12: Partie de code

En comparant la pose finale mesurée sur le robot réel avec la consigne opérationnelle initialement demandée, nous constatons un léger écart de positionnement géométrique. Cette divergence, inhérente à la robotique expérimentale, met en évidence les limites d'un modèle purement théorique face aux contraintes physiques réelles. Cet écart s'explique principalement par le jeu mécanique au niveau des réducteurs des moteurs Dynamixel, la flexion structurelle des bras sous l'effet de la gravité, ainsi que par les imperfections géométriques d'assemblage de la maquette. Néanmoins, l'exploitation du modèle géométrique inverse a permis de valider avec succès une configuration articulaire

exploitable , assurant un positionnement stable tout en respectant scrupuleusement les butées logicielles de sécurité établies pour préserver les actionneurs.

Boucle permettant d’afficher à l’écran en temps réel le mouvement du robot :

```
while any(maquette.isMoving([1, 2, 3]))

    p_c = maquette.getStepPosition([1, 2, 3]);
    q_c = step2angle(p_c, 512);
    modele.setAngularPosition(q_c);
    T_courant = modele.mod_geo_dir();
    x_c = T_courant(1, 4);
    y_c = T_courant(2, 4);
    modele.plot(0, 'red', 1);
    pause(0.05);

end

p_reel = maquette.getStepPosition([1, 2, 3]);
q_reel = step2angle(p_reel, 512);
modele.setAngularPosition(q_reel);
T_reel = modele.mod_geo_dir();
```

Figure 13: Partie de code

2.5 Génération de trajectoires

On cherche à réaliser une trajectoire par interpolation linéaire entre deux configurations opérationnelles :

Justification du choix des configurations

Pour réaliser l'interpolation linéaire, nous avons sélectionné des points de départ et d'arrivée situés au cœur de l'espace de travail utile du robot ($x_0 = 0.08$ m, $y_0 = 0.12$ m vers $x_f = 0.16$ m, $y_f = 0.12$ m). Ce choix garantit que le segment rectiligne reliant les deux poses ne traverse aucune zone de singularité géométrique et n'impose pas d'extensions extrêmes forçant les actionneurs au-delà de leurs butées mécaniques ($50 \leq \text{step} \leq 950$).

Critère de choix de la solution du Modèle Géométrique Inverse (MGI)

La fonction `mod_geo_inv` renvoie deux solutions distinctes pour chaque coordonnée opérationnelle intermédiaire. Pour assurer un mouvement fluide, nous avons implémenté le critère de la distance articulaire minimale (continuité de posture). À chaque point i , l'algorithme compare la distance euclidienne normée entre la configuration articulaire précédente q_{prec} et les deux nouvelles solutions proposées (q_1 et q_2). La solution sélectionnée est celle qui minimise cette distance : $\min(\|q_1 - q_{\text{prec}}\|, \|q_2 - q_{\text{prec}}\|)$. Ce critère évite un changement brutal de posture (inversion du coude) en plein milieu de la ligne droite, ce qui endommagerait la maquette ou dépasserait les butées.

Analyse de l'influence du nombre de points intermédiaires (n)

$n = 5$:

Lorsque le nombre de points intermédiaires est trop faible, le déplacement de l'organe terminal entre chaque consigne successive reste géré de manière indépendante par les boucles de position internes des servomoteurs Dynamixel. La trajectoire de l'outil subit des déviations courbes visibles entre deux points, et le mouvement global de la maquette est saccadé.

$n = 50$:

En augmentant le nombre de points à $n=50$, le découpage de la trajectoire devient extrêmement fin. Le robot reçoit des consignes très rapprochées dans l'espace opérationnel, ce qui force l'organe terminal à suivre de manière quasi parfaite la ligne droite idéale. Le mouvement réel devient nettement plus fluide et rectiligne.

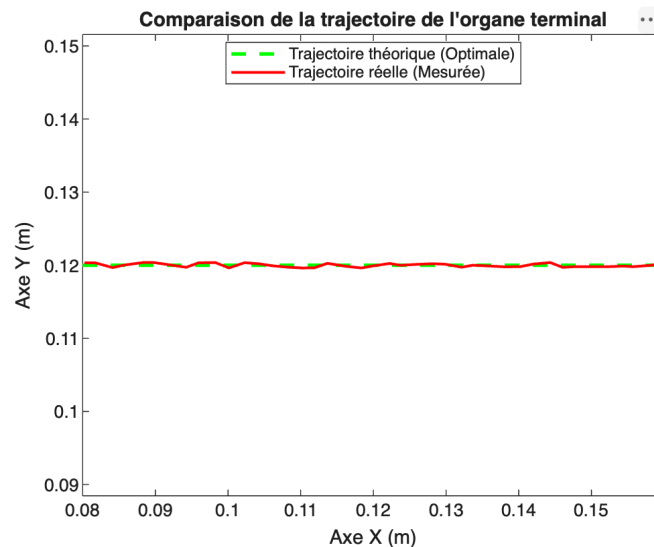


Figure 14: Comparaison des trajectoires

Ce graphique montre que le robot suit fidèlement la ligne droite théorique (pointillés verts). La courbe réelle (rouge) présente toutefois de légères micro-irrégularités, qui sont directement engendrées par les vibrations des bras en mouvement et les petites oscillations des moteurs réels.

2.6 Conclusion

Ce TP a permis de valider avec succès la mise en œuvre des modèles géométriques (direct et inverse) pour le contrôle d'un robot sériel 3R plan. L'implémentation de l'interpolation linéaire dans l'espace opérationnel a démontré son efficacité en forçant l'organe terminal à suivre fidèlement la trajectoire rectiligne théorique programmée. Les légers écarts observés sur le tracé final ne remettent pas en cause la validité des modèles calculés, mais illustrent

parfaitement les limites de la simulation face à la réalité physique de la maquette, ces micro-irrégularités étant directement engendrées par la vibration des bras en mouvement et les oscillations des moteurs réels.

3. Conclusion

Ces deux séances de travaux pratiques ont permis de couvrir l'intégralité de la chaîne de contrôle d'un robot sériel 3R plan, depuis le comportement physique des actionneurs jusqu'à la modélisation géométrique globale du bras.

Dans un premier temps, l'étude des servomoteurs a mis en évidence l'efficacité de leur asservissement interne tout en révélant les limites du traitement des signaux. Nous avons notamment pu observer l'impact du bruit de quantification numérique : le calcul de la vitesse par différence finie fait ressortir de fortes micro-oscillations dues à la mesure de position en pas entiers discrets, ce qui justifie l'utilisation de filtres ou de capteurs internes embarqués pour obtenir un signal fluide.

Dans un second temps, la mise en œuvre des modèles géométriques a permis de lier le monde informatique à l'espace réel. Le Modèle Géométrique Direct (MGD) a validé notre capacité à localiser le bout du bras (x,y) à partir des angles des moteurs, tandis que le Modèle Géométrique Inverse (MGI) s'est avéré indispensable pour traduire une position cible en consignes angulaires exécutables par les moteurs. La correspondance entre les simulations MATLAB et les mouvements de la maquette valide l'ensemble des modèles théoriques et démontre les exigences de précision nécessaires au pilotage d'un robot articulé.